



Carrera: Ingeniería en Sistemas.

Materia: Programación

Sección: Domingos, Sección B.

Ingeniero: David Alvarez

TEMA: Recorridos Manuales Grafos

Maura Selena Velasquez Ortega
9941 24 14056

Jimmy Aroldo Morales Carranza

9941 24 8021

Rony Alexander Barrios Borrayo

9941 99 127

Instrucciones



Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas

Nombre:	Maura Selena Velásquez Ortega	Carné:	9941-24-14056
Nombre:	Rony Alexander Barrios Barrios	Carné:	9941 99 127
Nombre:	Jimmy Ardo Nolasco Cervera	Carné:	

Carrera	Ingeniería en Sistemas	Carrera	9941
Asignatura	Programación III	Curso	022
Ciclo	Quinto Ciclo	Jornada	Domingos
Catedrático	Ing. MBA David Álvarez	Fecha	24/05/2026
Semestre	Primer Semestre	Sección	B

Objetivo
Aplicar recorridos de grafos mediante BFS y DFS, representando gráficamente los grafos, resolviendo recorridos manualmente y comparando una implementación en C++ puro contra una implementación en Java con Collections Framework.

Parte 1 – Recorridos manuales de grafos

Instrucciones
Para cada ejercicio, el grupo debe:

1. Dibujar el grafo.
2. Escribir la lista de adyacencia.
3. Realizar el recorrido BFS desde el nodo indicado.
4. Realizar el recorrido DFS desde el nodo indicado.
5. Indicar el orden de visita.

Cuando exista más de un vecino disponible, deben visitar los nodos en orden ascendente o alfabético.

Ejercicio 1 1-2 1-3 2-4 3-5 Inicio: 1	Ejercicio 2 1-2 1-4 2-3 4-5 5-6 Inicio: 1	Ejercicio 3 A-B A-C B-D B-E C-F Inicio: A
Ejercicio 4 1-2 2-3 3-4 4-5 Inicio: 1	Ejercicio 5 1-2 1-3 1-4 2-5 4-6 Inicio: 1	Ejercicio 6 A-B B-C C-D A-E E-F Inicio: A
Ejercicio 7 1-2 1-3 2-4 2-5 3-6 3-7 Inicio: 1	Ejercicio 8 A-B A-C B-D C-D D-E Inicio: A	Ejercicio 9 1-2 2-3 3-1 3-4 4-5 Inicio: 1
Ejercicio 10 A-B A-D B-C D-E C-F E-F Inicio: A	Ejercicio 11 1-2 1-5 2-3 2-4 5-6 6-7 Inicio: 1	Ejercicio 12 A-B A-C B-E C-D D-E E-F Inicio: A
Ejercicio 13 1-2 1-3 2-4 4-6 3-5 5-6 6-7 Inicio: 1	Ejercicio 14 A-B B-C C-D D-A A-E E-F Inicio: A	Ejercicio 15 1-2 1-3 2-5 3-4 4-6 5-6 6-7 7-8 Inicio: 1
Ejercicio 16 1-2 1-3 2-4 2-5 3-6 5-7 6-7 7-8 8-9 Inicio: 1	Dibuje el grafo. Escriba la lista de adyacencia. Realice el recorrido BFS desde el nodo 1. Realice el recorrido DFS desde el nodo 1. Responda: • ¿En qué momento (nivel) BFS llega al nodo 9? • ¿DFS visita el nodo 9 antes o después que BFS? • ¿Cuál recorrido considera más eficiente en este caso y por qué?	

Parte 2 – Comparación técnica C++ vs Java

Instrucciones

- El grupo deberá ejecutar dos implementaciones funcionales del mismo grafo:
- Una versión en C++ puro, sin usar vector, queue, stack, map ni unordered_map.
 - Una versión en Java, usando HashMap, ArrayList, Queue y LinkedList.
- Luego deberán elaborar una comparación técnica.

Código en C++



Código en Java



Análisis técnico requerido

El grupo debe responder:

- ¿Qué estructuras se implementaron manualmente en C++?
- ¿Qué estructuras ya existen en Java?
- ¿Cuál código es más largo?
- ¿Cuál código es más fácil de entender?
- ¿Cuál lenguaje exige mayor control del programador?
- ¿Qué ventajas tiene C++ en este caso?
- ¿Qué ventajas tiene Java en este caso?
- ¿Qué sucede si en C++ no se libera memoria correctamente?
- ¿Por qué BFS necesita una cola?
- ¿Por qué DFS puede resolverse con recursión?

Parte 3 – Video tipo entrevista

Cada grupo debe grabar un video de 5 a 8 minutos.

Formato sugerido

Debe aparecer al menos una vez cada integrante.

El video debe responder:

- ¿Qué es BFS?
- ¿Qué es DFS?
- ¿Cuál fue la diferencia más importante entre ambos recorridos?
- ¿Qué parte fue más fácil?
- ¿Qué parte fue más difícil?
- ¿Qué diferencias encontraron entre C++ y Java?
- ¿Por qué en C++ se trabajó sin estructuras ya hechas?
- ¿Cuál implementación consideran más útil para desarrollo real?

Entregables

El grupo debe entregar:

- Documento PDF con:
 - Integrantes
 - Dibujos de los 15 grafos
 - Lista de adyacencia de cada grafo
 - Recorrido BFS
 - Recorrido DFS
 - Análisis técnico C++ vs Java
- Repositorio GitHub con:
 - Carpeta /cpp
 - Carpeta /java
 - Código funcional
 - Capturas de ejecución
- Video:
 - Link de Google Drive, YouTube no listado o similar
 - Duración: 5 a 8 minutos

Rúbrica – 3 puntos

Criterio

Dibujos y listas de adyacencia de los 15 grafos

Recorridos BFS correctos

Recorridos DFS correctos

Ejecución y evidencia de código C++ y Java

Comparación técnica entre C++ y Java

Video tipo entrevista con participación del grupo

Total

Puntos

0.60

0.60

0.60

0.50

0.40

0.30

3.00

Recorridos manuales de grafos

Ejercicio 1



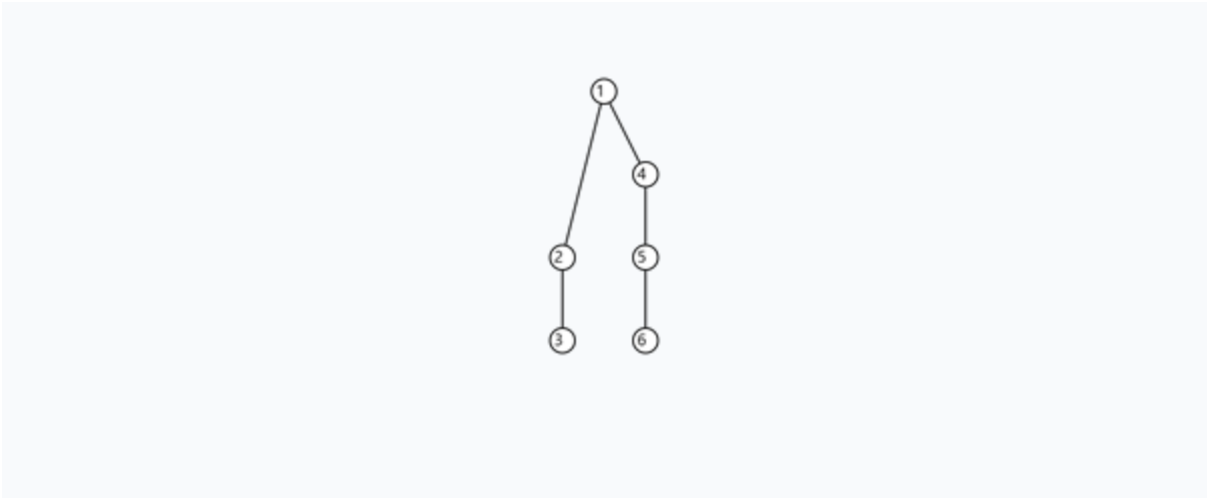
2. Lista de Adyacencia (Bidireccional y Ordenada)

```
{  
  "1": [  
    "2",  
    "3"  
  ],  
  "2": [  
    "1",  
    "4"  
  ],  
  "3": [  
    "1",  
    "5"  
  ],  
  "4": [  
    "2"  
  ],  
  "5": [  
    "3"  
  ]  
}
```

3. Secuencias (Origen: 1)

BFS	1 -> 2 -> 3 -> 4 -> 5
DFS	1 -> 2 -> 4 -> 3 -> 5

Ejercicio 2



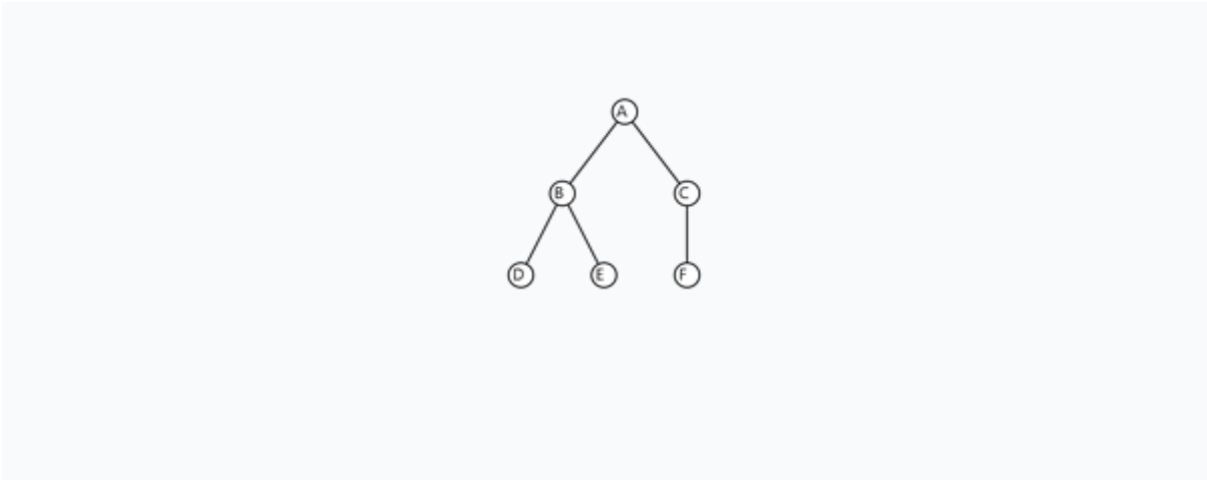
2. Lista de Adyacencia (Bidireccional y Ordenada)

```
{
  "1": [
    "2",
    "4"
  ],
  "2": [
    "1",
    "3"
  ],
  "3": [
    "2"
  ],
  "4": [
    "1",
    "5"
  ],
  "5": [
    "4",
    "6"
  ],
  "6": [
    "5"
  ]
}
```

3. Secuencias (Origen: 1)

BFS	1 -> 2 -> 4 -> 3 -> 5 -> 6
DFS	1 -> 2 -> 3 -> 4 -> 5 -> 6

Ejercicio 3



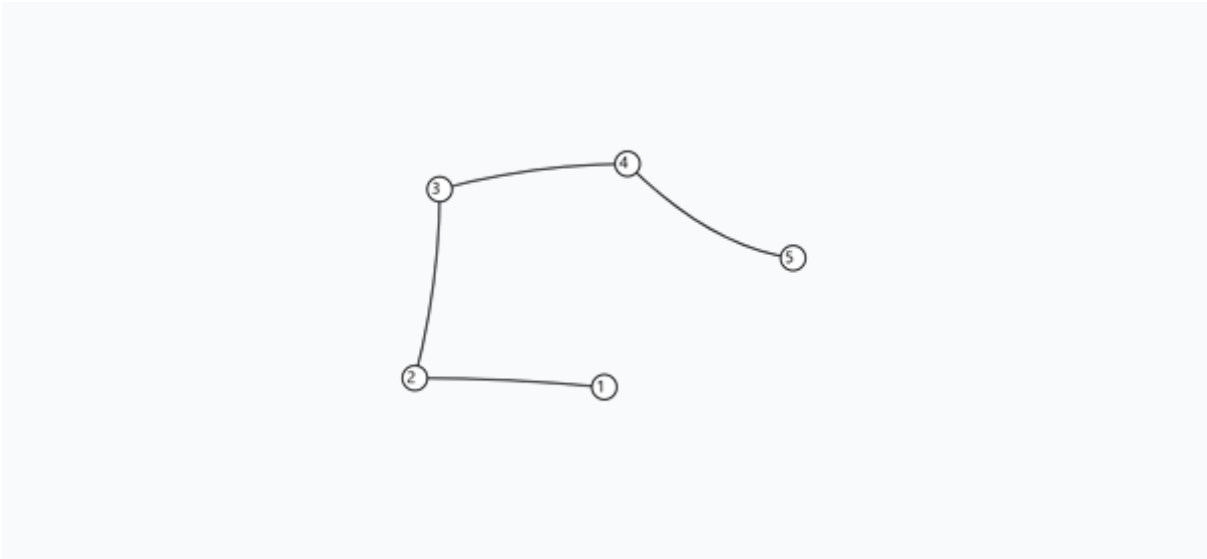
2. Lista de Adyacencia (Bidireccional y Ordenada)

```
{
  "A": [
    "B",
    "C"
  ],
  "B": [
    "A",
    "D",
    "E"
  ],
  "C": [
    "A",
    "F"
  ],
  "D": [
    "B"
  ],
  "E": [
    "B"
  ],
  "F": [
    "C"
  ]
}
```

3. Secuencias (Origen: A)

BFS	A -> B -> C -> D -> E -> F
DFS	A -> B -> D -> E -> C -> F

Ejercicio 4



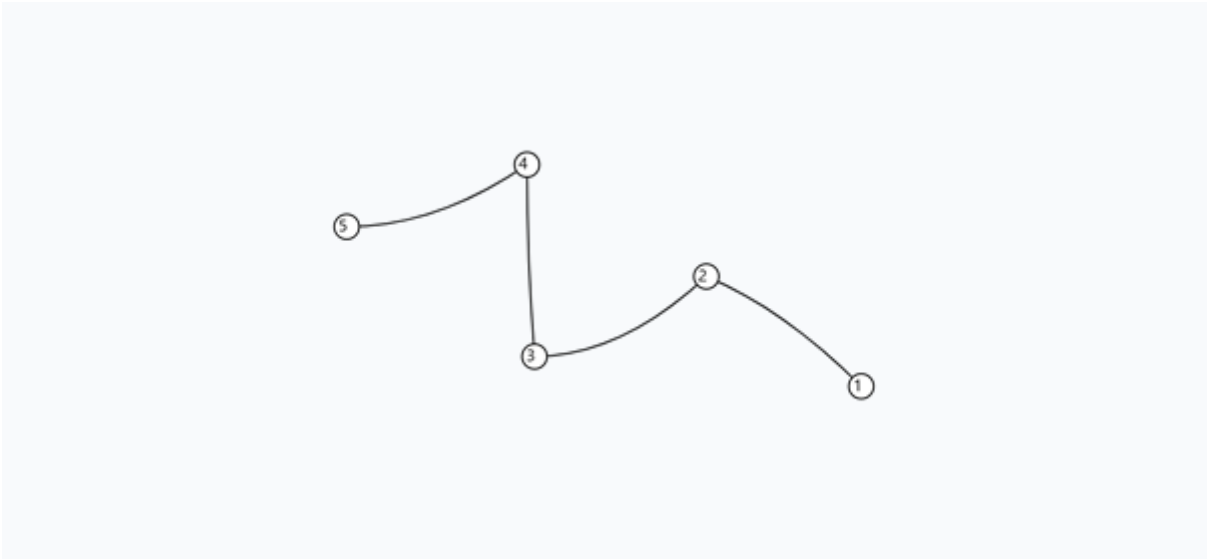
2. Lista de Adyacencia (Bidireccional y Ordenada)

```
{
  "1": [
    "2"
  ],
  "2": [
    "1",
    "3"
  ],
  "3": [
    "2",
    "4"
  ],
  "4": [
    "3",
    "5"
  ],
  "5": [
    "4"
  ]
}
```

3. Secuencias (Origen: 1)

BFS	1 -> 2 -> 3 -> 4 -> 5
DFS	1 -> 2 -> 3 -> 4 -> 5

Ejercicio 5



2. Lista de Adyacencia (Bidireccional y Ordenada)

```
{
  "1": [
    "2"
  ],
  "2": [
    "1",
    "3"
  ],
  "3": [
    "2",
    "4"
  ],
  "4": [
    "3",
    "5"
  ],
  "5": [
    "4"
  ]
}
```


3. Secuencias (Origen: 1)

BFS	1 -> 2 -> 3 -> 4 -> 5
DFS	1 -> 2 -> 3 -> 4 -> 5

Ejercicio 6



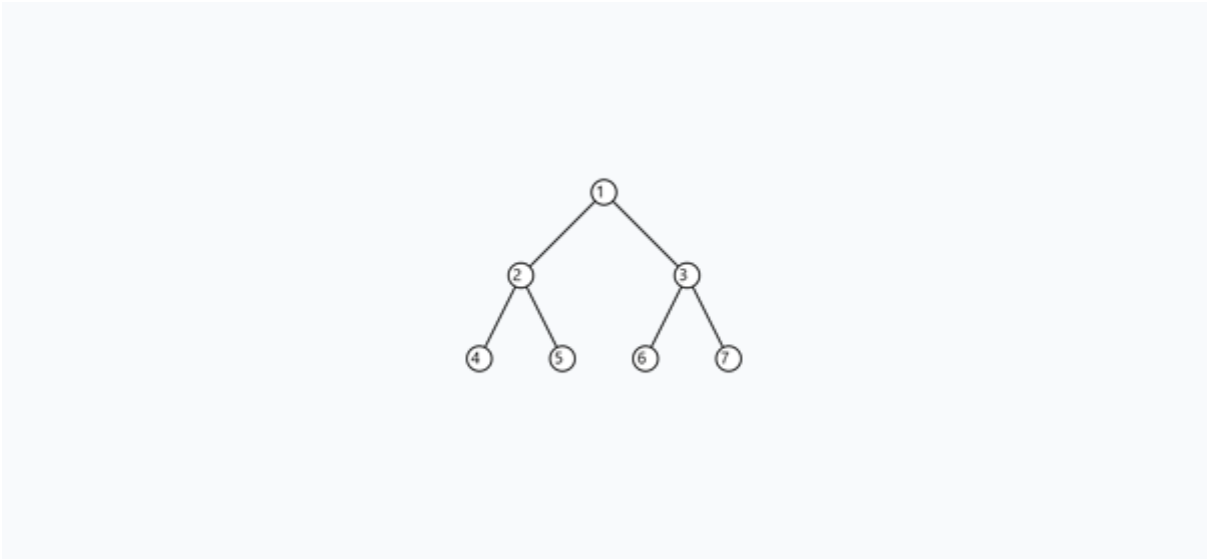
2. Lista de Adyacencia (Bidireccional y Ordenada)

```
{  
  "A": [  
    "B",  
    "E"  
  ],  
  "B": [  
    "A",  
    "C"  
  ],  
  "C": [  
    "B",  
    "D"  
  ],  
  "D": [  
    "C"  
  ],  
  "E": [  
    "A",  
    "F"  
  ],  
  "F": [  
    "E"  
  ]  
}
```

3. Secuencias (Origen: A)

BFS	A -> B -> E -> C -> F -> D
DFS	A -> B -> C -> D -> E -> F

Ejercicio 7



2. Lista de Adyacencia (Bidireccional y Ordenada)

```
{
  "1": [
    "2",
    "3"
  ],
  "2": [
    "1",
    "4",
    "5"
  ],
  "3": [
    "1",
    "6",
    "7"
  ],
  "4": [
    "2"
  ],
  "5": [
    "2"
  ],
  "6": [
    "3"
  ]
}
```

```

],
"7": [
  "3"
]
}

```

3. Secuencias (Origen: 1)

BFS	1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7
DFS	1 -> 2 -> 4 -> 5 -> 3 -> 6 -> 7

Ejercicio 8



2. Lista de Adyacencia (Bidireccional y Ordenada)

```

{
  "A": [
    "B",
    "C"
  ],
  "B": [
    "A",
    "D"
  ],
  "C": [
    "A",
    "D"
  ],
  "D": [
    "B",
    "C",
    "E"
  ]
}

```

```

],
"E": [
  "D"
]
}

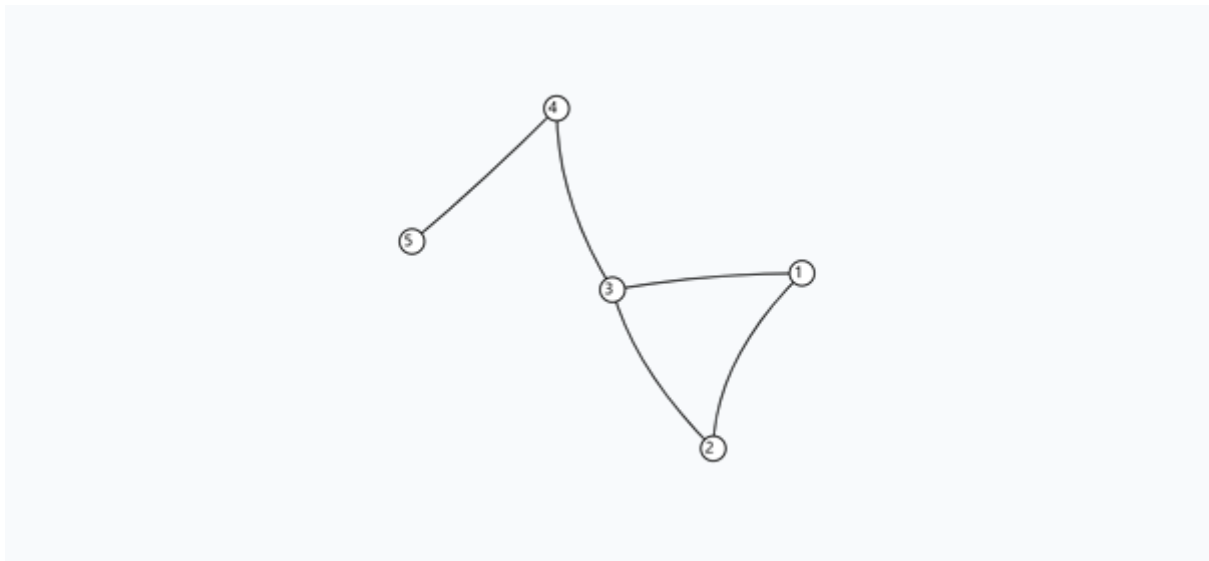
```

3. Secuencias (Origen: A)

BFS	A -> B -> C -> D -> E
DFS	A -> B -> D -> C -> E

Ejercicio 9

1. Representación Gráfica (No Dirigido)



2. Lista de Adyacencia (Bidireccional y Ordenada)

```

{
  "1": [
    "2",
    "3"
  ],
  "2": [
    "1",
    "3"
  ],
  "3": [
    "1",
    "2",
    "4"
  ],
  "4": [

```

```

    "3",
    "5"
  ],
  "5": [
    "4"
  ]
}

```

3. Secuencias (Origen: 1)

BFS	1 -> 2 -> 3 -> 4 -> 5
DFS	1 -> 2 -> 3 -> 4 -> 5

Ejercicio 10



2. Lista de Adyacencia (Bidireccional y Ordenada)

```

{
  "A": [
    "B",
    "D"
  ],
  "B": [
    "A",
    "C"
  ],
  "D": [
    "A",
    "E"
  ],
}

```



```

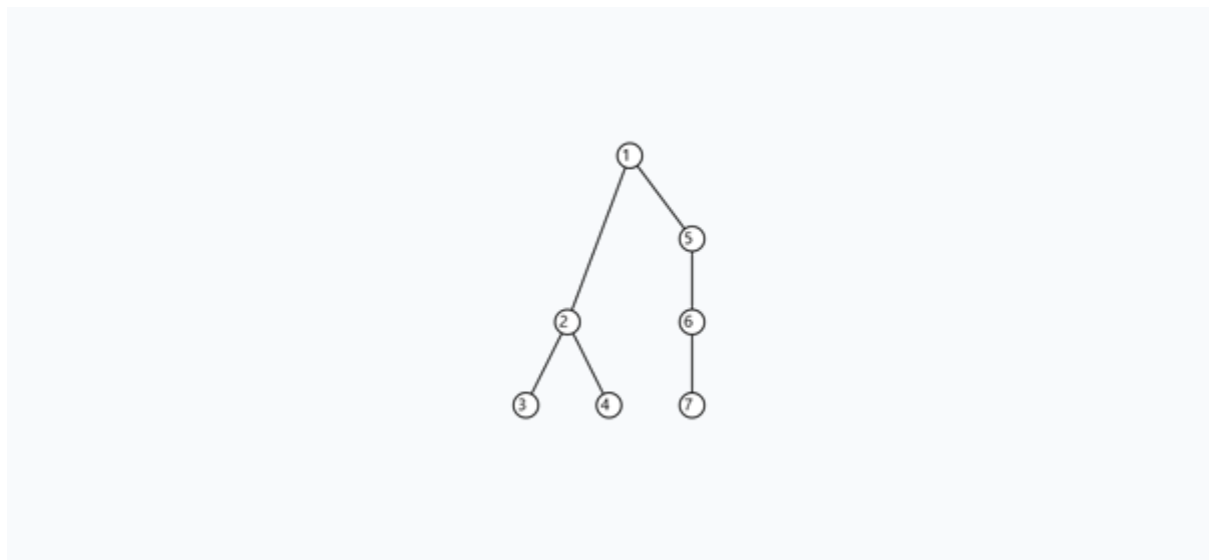
"C": [
    "B",
    "F"
],
"E": [
    "D",
    "F"
],
"F": [
    "C",
    "E"
]
}

```

3. Secuencias (Origen: A)

BFS	A -> B -> D -> C -> E -> F
DFS	A -> B -> C -> F -> E -> D

Ejercicio 11



2. Lista de Adyacencia (Bidireccional y Ordenada)

```

{
  "1": [
    "2",
    "5"
  ],
  "2": [
    "1",
    "3",
    "4"
  ]
}

```

```

],
"3": [
    "2"
],
"4": [
    "2"
],
"5": [
    "1",
    "6"
],
"6": [
    "5",
    "7"
],
"7": [
    "6"
]
}

```

3. Secuencias (Origen: 1)

BFS	1 -> 2 -> 5 -> 3 -> 4 -> 6 -> 7
DFS	1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7

Ejercicio 12



2. Lista de Adyacencia (Bidireccional y Ordenada)

```
{  
  "1": [  
    "2",  
    "5"  
  ],  
  "2": [  
    "1",  
    "3",  
    "4"  
  ],  
  "3": [  
    "2"  
  ],  
  "4": [  
    "2"  
  ],  
  "5": [  
    "1",  
    "6"  
  ],  
  "6": [  
    "5",  
    "7"  
  ],  
  "7": [  
    "6"  
  ]  
}
```

3. Secuencias (Origen: 1)

BFS	1 -> 2 -> 5 -> 3 -> 4 -> 6 -> 7
DFS	1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7

Ejercicio 13



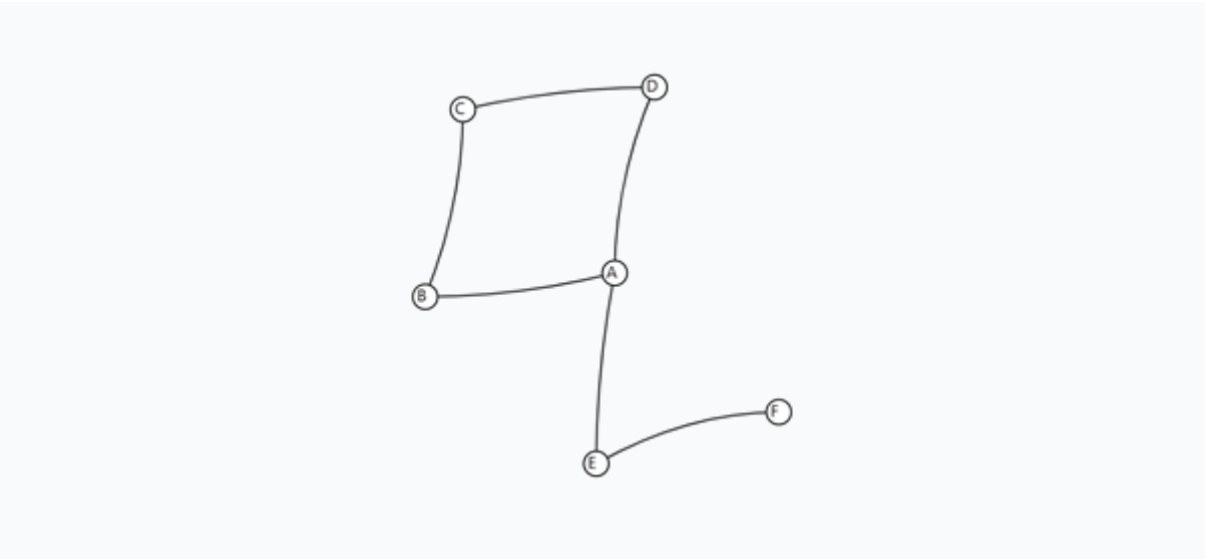
2. Lista de Adyacencia (Bidireccional y Ordenada)

```
{  
  "1": [  
    "2",  
    "3"  
  ],  
  "2": [  
    "1",  
    "4"  
  ],  
  "3": [  
    "1",  
    "5"  
  ],  
  "4": [  
    "2",  
    "6"  
  ],  
  "5": [  
    "3",  
    "6"  
  ],  
  "6": [  
    "4",  
    "5",  
    "7"  
  ],  
  "7": [  
    "6"  
  ]  
}
```

3. Secuencias (Origen: 1)

BFS	1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7
DFS	1 -> 2 -> 4 -> 6 -> 5 -> 3 -> 7

Ejercicio 14



2. Lista de Adyacencia (Bidireccional y Ordenada)

```
{
  "A": [
    "B",
    "D",
    "E"
  ],
  "B": [
    "A",
    "C"
  ],
  "C": [
    "B",
    "D"
  ],
  "D": [
    "A",
    "C"
  ],
  "E": [
    "A",
    "F"
  ],
  "F": [
    "E"
  ]
}
```

```

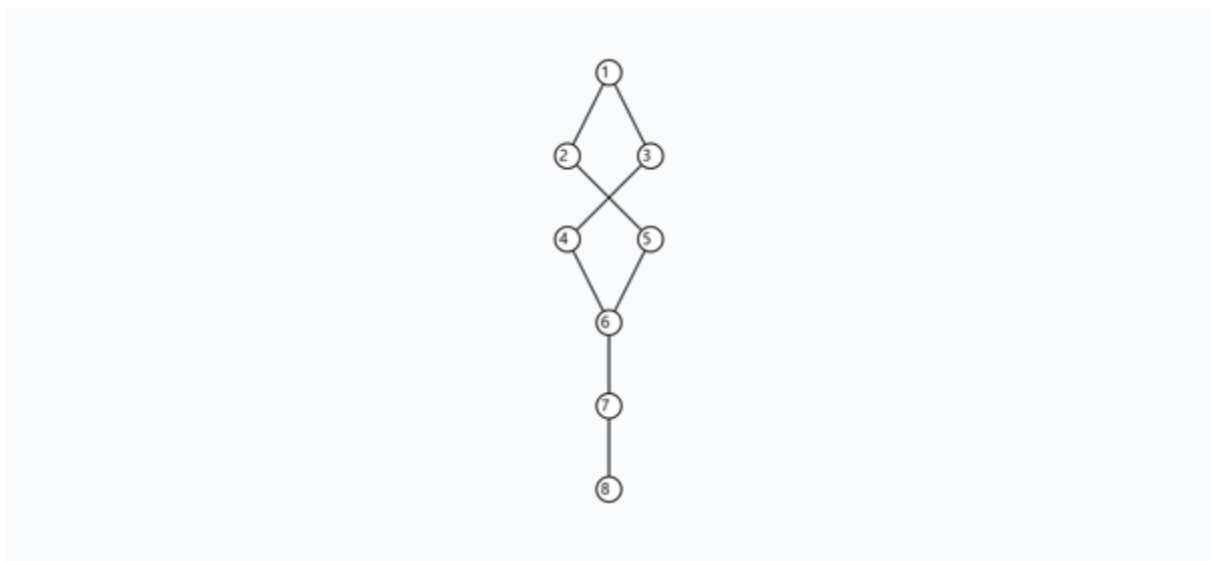
    "E"
  ]
}

```

3. Secuencias (Origen: A)

BFS	A -> B -> D -> E -> C -> F
DFS	A -> B -> C -> D -> E -> F

Ejercicio 15



2. Lista de Adyacencia (Bidireccional y Ordenada)

```

{
  "1": [
    "2",
    "3"
  ],
  "2": [
    "1",
    "5"
  ],
  "3": [
    "1",
    "4"
  ],
  "4": [
    "3",
    "6"
  ],
  "5": [
    "2",

```

```

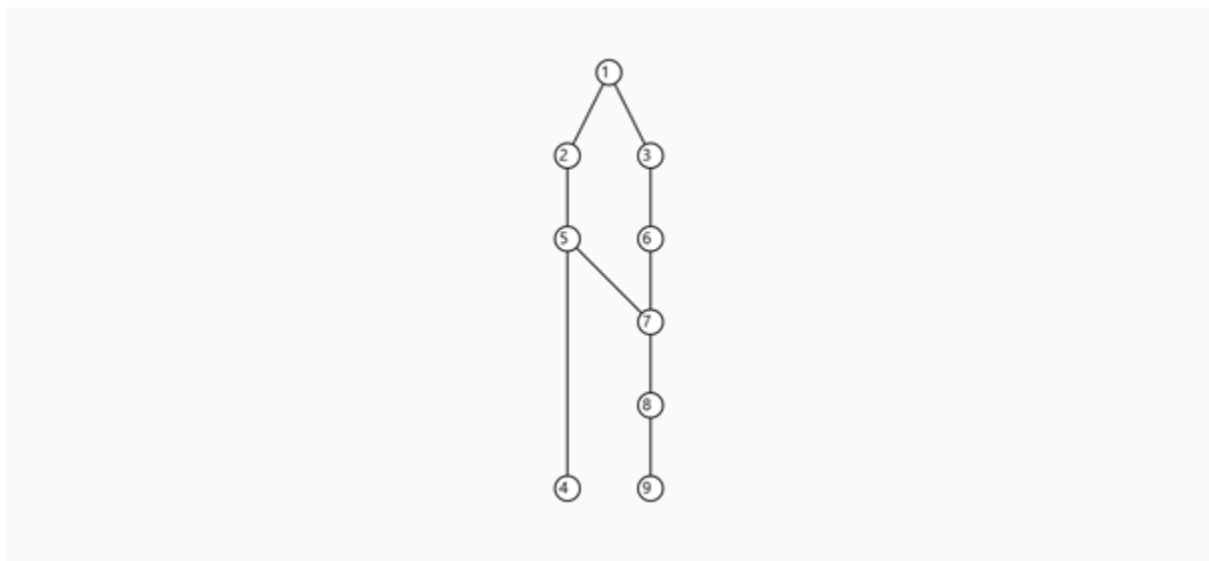
        "6"
    ],
    "6": [
        "4",
        "5",
        "7"
    ],
    ],
    "7": [
        "6",
        "8"
    ],
    ],
    "8": [
        "7"
    ]
}

```

3. Secuencias (Origen: 1)

BFS	1 -> 2 -> 3 -> 5 -> 4 -> 6 -> 7 -> 8
DFS	1 -> 2 -> 5 -> 6 -> 4 -> 3 -> 7 -> 8

Ejercicio 16



2. Lista de Adyacencia (Bidireccional y Ordenada)

```

{
  "1": [
    "2",
    "3"
  ],
  "2": [
    "1",

```

```

        "4",
        "5"
    ],
    "3": [
        "1",
        "6"
    ],
    "4": [
        "2"
    ],
    "5": [
        "2",
        "7"
    ],
    "6": [
        "3",
        "7"
    ],
    "7": [
        "5",
        "6",
        "8"
    ],
    "8": [
        "7",
        "9"
    ],
    "9": [
        "8"
    ]
}

```

3. Secuencias (Origen: 1)

BFS	1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9
DFS	1 -> 2 -> 4 -> 5 -> 7 -> 6 -> 3 -> 8 -> 9

¿En qué momento (nivel) BFS llega al nodo 9?

El grafo alcanza un Nivel 5 (Raíz Nivel 0). Hojas profundas: [9].

¿DFS visita el nodo 9 antes o después que BFS?

Al existir más de un vecino, se visitan obligatoriamente en estricto orden ascendente natural (números primero, luego letras).

¿Cuál recorrido considera más eficiente en este caso y por qué?

BFS: elije **3** resolviendo horizontalmente el nivel.

DFS: elije **4** penetrando verticalmente la rama.

Parte 3

Preguntas de la entrevista :

1. ¿Qué es BFS?

BFS (Búsqueda en Anchura) es un algoritmo de exploración de grafos que avanza **nivel por nivel**. Comenzando desde un nodo inicial, el algoritmo visita primero a todos sus vecinos inmediatos (Nivel 1), luego a los vecinos de estos (Nivel 2), y así sucesivamente. Para garantizar este orden horizontal y procesar los elementos en orden estricto de llegada, utiliza una estructura de **Cola** bajo la lógica **FIFO** (*First In, First Out*).

2. ¿Qué es DFS?

DFS (o Búsqueda en Profundidad) es un algoritmo que toma el enfoque opuesto a BFS: explora un camino **lo más profundo posible** antes de retroceder (*backtracking*). En lugar de abrirse hacia los lados, el algoritmo se mueve de forma vertical hacia el fondo de una rama; cuando llega a un punto ciego, regresa al último nodo con vecinos sin visitar. Requiere una estructura de **Pila** bajo la lógica **LIFO** (*Last In, First Out*), la cual suele implementarse de forma automática mediante **recursión**.

3. ¿Cuál fue la diferencia más importante entre ambos recorridos?

La diferencia fundamental radica en la **estrategia de exploración** y la **estructura de datos** que los respalda:

- **BFS** explora de forma expansiva y horizontal usando una **Cola**, lo que lo hace ideal para encontrar la ruta más corta en grafos no ponderados.
- **DFS** explora de forma lineal y vertical usando una **Pila**, siendo óptimo para tareas que requieren revisar todas las combinaciones posibles, como resolver laberintos o juegos de lógica.

4. ¿Qué parte fue más fácil?

La parte más sencilla fue el análisis y ejecución de la versión en **Java**. Gracias al *Collections Framework*, este lenguaje ofrece herramientas nativas listas para usarse como **HashMap**, **ArrayList** y **LinkedList**. Esto permite que el código sea limpio, directo y fácil de comprender, ya que no requiere configurar la lógica interna de almacenamiento ni preocuparse por la destrucción manual de los objetos.

5. ¿Qué parte fue más difícil?

Diríamos que la sección más compleja fue la comprensión de la versión en **C++ puro**. Al no contar con estructuras dinámicas prefabricadas (como `std::vector` o `std::queue`), se debe gestionar el grafo mediante **punteros y manejo manual de memoria dinámica**. Conectar cada nodo manualmente en la memoria RAM y asegurar la correcta liberación de la misma con `delete` incrementa notablemente la dificultad y el riesgo de errores lógicos.

6. ¿Qué diferencias encontraron entre C++ y Java?

- **Gestión de Memoria:** En C++ la asignación y liberación de memoria es responsabilidad absoluta del programador. En Java, el **Recolector de Basura** o también conocido como (*Garbage Collector*) automatiza este proceso, eliminando el riesgo de colapsar el sistema por descuidos de memoria.
- **Eficiencia vs. Comodidad:** C++ trabaja a bajo nivel, lo que elimina la sobrecarga de procesos intermedios, ofreciendo una velocidad y control directo sobre el hardware inigualables. Java sacrifica una pequeña fracción de ese rendimiento a cambio de agilidad de desarrollo, legibilidad y seguridad del código.

7. ¿Por qué en C++ se trabajó sin estructuras ya hechas?

Se trabajó de esta manera con un **propósito puramente didáctico**. Utilizar estructuras prefabricadas en Java facilita el trabajo, pero oculta la lógica real. Programar un grafo desde cero en C++ obliga a comprender cómo interactúan los punteros en la memoria RAM, cómo se estructuran físicamente las listas de adyacencia y cuál es el costo real de procesamiento detrás de cada operación. En pocas palabras, enseña cómo funcionan los datos "bajo el capó".

8. ¿Cuál implementación consideran más útil para desarrollo real?

- **Java** es la opción más útil para la gran mayoría de **aplicaciones comerciales, plataformas empresariales y servicios web modernos**. En entornos donde el tiempo de desarrollo, la facilidad de mantenimiento y la seguridad contra fallos lógicos son prioritarios, las librerías nativas de Java ofrecen una ventaja competitiva enorme.
- **C++** sigue siendo la opción indiscutible únicamente en **sistemas críticos de alto rendimiento**, tales como motores de videojuegos, sistemas operativos, sistemas embebidos o software científico, donde exprimir el hardware al máximo es más importante que la velocidad con la que se escribe el código.